

Chapter 9 — The agent

Working with a system that acts

Session 1 was a system that answers. This one edits your files.



Start where the barrier is lowest

- Desktop app and the editor extension first. The terminal is optional and it is taught last.
- A browser fallback exists for a machine that fights the install. Nobody in this room should lose the afternoon to a package manager.
- The terminal is not the skill. The skill is everything on the following slides, and it is identical in all three.

Terminal-first teaching filters for people who already have the tool. This room does not, and the filter would remove the audience rather than the difficulty.



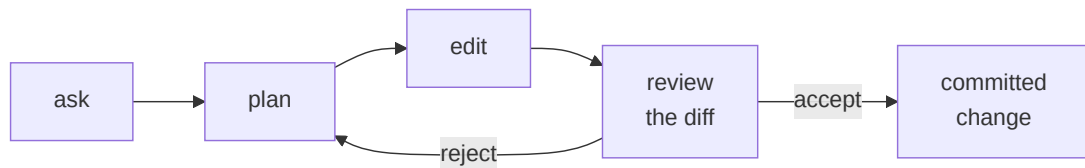
The permission model is the safety story

- Read-only by default. Nothing is written until you say so.
- Every proposed edit is shown as a **diff** before it is applied.
- Permissions are per action, and can be narrowed or broadened per project.

The **diff** is the artefact of responsibility. It is the moment the change becomes yours. Approving a diff you have not read is the whole risk of this chapter in one action.



The working loop



- **Plan before edit.** Plan mode produces the intended change as text, before a single file is touched, and it is far cheaper to reject a plan than a patch.
- The loop is the same whether the task takes one turn or forty.



Context engineering is the skill

Not a footnote. This is the single thing that separates competent agent use from incompetent agent use.

- Everything in the session shares one bounded window — your files, the outputs, the tool results, the conversation.
- It fills. Chapter 5 told you what happens to accuracy as it fills, and Chapter 7 told you what it costs.
- Nothing warns you at the moment it starts mattering.

Thread — memory and context. Introduced in Chapter 1 as a bounded buffer, watched degrading in Chapter 5, named as the missing hippocampus in Chapter 8. This is where you manage it. Chapter 10 is where you replace it.



Three moves, and knowing which one

- **Compact** — summarise the session so far and continue. Cheap *relative to continuing on a full window*, but not free: the summarising call itself reads the whole context, which by Chapter 7 is the most expensive call available at that moment. Lossy, and the loss is invisible.
- **Clear** — discard and start fresh in the same project. Loses everything not written to a file.
- **Abandon** — start a new session because this one has gone wrong in a way summarising will not fix.

The third is the one people will not do, because the session feels expensive to lose. It is not. It has already cost you the thing you were protecting.



Demonstration: what contamination looks like

- Run one task in a fresh session, **three times**. Record the answers.
- Run the same task after a long, unrelated conversation in the same session, **three times**. Record those.
- Put the two sets side by side and look at the spread, not at one pair.

TODO(capture) — instructor records both sets in advance, with model ID and date.

Say what this is. Six runs is an **illustration**, not a measurement — it is under the floor Chapter 4 set, and Chapter 4 is the reason you know that. Running one pair and declaring a difference would be the exact design that chapter spent forty minutes forbidding. If the two sets overlap completely, say so; that is a result too.



Session commands worth knowing on day one

- `/doctor` — a setup checkup that diagnoses, and can fix, installation and configuration problems.
`/checkup` is an alias for it. Reach for it when behaviour is odd, *before* you start debugging the thing you were working on.
- Check your usage early in the session, not when you hit a limit mid-demonstration.
- Resume a previous session when you need the reasoning behind a result, not just the result.

Version-fragile — and now there is something to hedge. Command names, flags and behaviours change between releases; `/doctor` was extended in a recent version to audit memory files and unused components. These are correct as recorded and get re-fetched in the week before the session, per `DECISIONS.md` item 6.



Exercise

Point it at one of your own MATLAB scripts.

1. Ask for an explanation of what the script does. Read it against what you know the script does.
2. Ask for **one small reversible change**.
3. Read the diff before accepting it. Say out loud what the change does.
4. Reject it once, on purpose, and watch what happens.

Step 4 is not padding. Most people never reject anything, and then discover the reject path during something that matters.



Where this leaves us

- A system that acts, with a permission model and a diff as the point of responsibility.
- A bounded window you are now actively managing rather than passively filling.
- Three moves, and the discipline to use the third.

Everything here still dies with the session. Chapter 10 is where you stop losing it.

